Université Abdelmalek Essaâdi
ENSA Al Hoceima

Compte Rendu : TP N°3

Java SE (JSE) - ID1

Réalisé par :
Hidaya El hichami

Présenté à :
Pr. Mohamed CHERRADI

Table des matières

1 Introduction **3**

2 Exercice 1 : Personne / Etudiant **3**
2.1 Énoncé . . . 3
2.2 Personne.Java : . . . 3
2.3 Etudiant.java : . . . 4
2.4 Main.java : . . . 4
2.5 Résultat d'exécution : . . . 5

3 Exercice 2 : Vehicule / Voiture **5**
3.1 Énoncé : . . . 5
3.2 Vehicule.java : . . . 5
3.3 Voiture.java : . . . 6
3.4 Main.java : . . . 6
3.5 Résultat d'exécution : . . . 7

4 Exercice 3 : Système bancaire **7**
4.1 Énoncé : . . . 7
4.2 Compte.java : . . . 8
4.3 CompteCourant.java : . . . 8
4.4 CompteEpargne.java : . . . 9
4.5 Main.java : . . . 9
4.6 Résultat d'exécution : . . . 10

5 Exercice 4 : Polymorphisme - Animaux **10**
5.1 Énoncé : . . . 10
5.2 Animal.java : . . . 11
5.3 Chat.java : . . . 11
5.4 Chien.java : . . . 11
5.5 Main.java : . . . 12
5.6 Résultat d'exécution : . . . 13

6 Exercice 5 : Polymorphisme - Figures Géométriques **13**
6.1 Énoncé : . . . 13
6.2 Figure.java : . . . 13
6.3 Cercle.java : . . . 14
6.4 Triangle.java : . . . 14
6.5 Main.java : . . . 15
6.6 Résultat d'exécution : . . . 16

7 Exercice 7 : Classe abstraite et calculs géométriques **16**
7.1 Énoncé : . . . 16
7.2 Forme.java : . . . 16
7.3 Cercle.java : . . . 17
7.4 Rectangle.java : . . . 17
7.5 Main.java : . . . 18
7.6 Résultat d'exécution : . . . 19

8 Exercice 8 : Interface **19**
8.1 Énoncé : 19
8.2 Volant.java : 19
8.3 Avion.java : 19
8.4 Oiseau.java : 20
8.5 Main.java : 20
8.6 Résultat d'exécution : 21

9 Exercice 9 : Héritage, Interface et Abstraction **21**
9.1 Énoncé : 22
9.2 Vehicule.java : 22
9.3 Electrique.java : 22
9.4 VoitureElectrique.java : 22
9.5 Moto.java : 23
9.6 Main.java : 24
9.7 Résultat d'exécution : 24

10 Exercice 10 : Système de Gestion de Bibliothèque (Synthèse) **25**
10.1 Énoncé : 25
10.2 Document.java : 25
10.3 Empruntable+Consultable.java : 26
10.4 Livre.java : 26
10.5 Magazine.java : 27
10.6 DocumentNumerique.java : 27
10.7 Utilisateur.java : 28
10.8 Main.java : 28
10.9 Résultat d'exécution : 29

11 Conclusion : **29**

1 Introduction

Ce TP a pour objectif de comprendre les concepts fondamentaux de la programmation orientée objet (POO), notamment l'héritage, le polymorphisme, les classes abstraites et les interfaces.

2 Exercice 1 : Personne / Etudiant

Cet exercice introduit le concept d'héritage en programmation orientée objet à travers la spécialisation d'une classe **Personne** en une classe **Etudiant**.

2.1 Énoncé

On souhaite modéliser des individus et plus particulièrement des étudiants.

- La classe **Personne** contient :
  - les attributs : nom, prénom, âge
  - des méthodes pour afficher les informations, vérifier la majorité et se présenter
- La classe **Etudiant** hérite de **Personne** et ajoute :
  - niveau d'étude et moyenne
  - calcul de la mention
  - vérification d'admission

2.2 Personne.Java :

```java
package Ex1;

public class Personne {
    String nom;
    String prenom;
    int age;

    public Personne(String nom, String prenom, int age) {
        this.nom = nom;
        this.prenom = prenom;
        this.age = age;
    }

    public void afficherInformations() {
        System.out.println("Nom: " + nom);
        System.out.println("Pr nom: " + prenom);
        System.out.println("Age: " + age);
    }

    public boolean estMajeur() {
        return age >= 18;
    }

    public String sePresenter() {
        return "je m'appelle " + prenom + " " + nom + " et j'ai " + age
            + " ans.";
    }
```

```
}
```

2.3 Etudiant.java :

```java
package Ex1;

public class Etudiant extends Personne {

    String niveau;
    double moyenne;

    public Etudiant(String nom, String prenom, int age, String niveau,
       double moyenne) {
        super(nom, prenom, age);
        this.niveau = niveau;
        this.moyenne = moyenne;
    }

    public String calculerMention() {
        if (moyenne >= 16)
            return "Tr s bien";
        else if (moyenne >= 14)
            return "Bien";
        else if (moyenne >= 12)
            return "Assez bien";
        else if (moyenne >= 10)
            return "Passable";
        else
            return "Aucune mention";
    }

    public boolean estAdmis() {
        return moyenne >= 10;
    }

    public void afficherProfil() {
        afficherInformations();
        System.out.println("Niveau: " + niveau);
        System.out.println("Moyenne: " + moyenne);
        System.out.println("Mention: " + calculerMention());
        System.out.println("Admis: " + (estAdmis() ? "Oui" : "Non"));
    }
}
```

2.4 Main.java :

```java

package Ex1;

public class Main {
    public static void main(String[] args) {
```

```java

        Etudiant e1 = new Etudiant("El Hichami", "Hidaya", 20, "2eme
            annee id", 19);

        System.out.println(e1.sePresenter());
        System.out.println("Majeur: " + e1.estMajeur());

        System.out.println("Le profil de l'etudiant:");
        e1.afficherProfil();
    }
}
```

2.5 Résultat d'exécution :

```
je m'appelle Hidaya El Hichami et j'ai 20ans.
Majeur: true
Le profil de L'etudiant:
Nom: El Hichami
Prénom: Hidaya
Age: 20
Niveau:2ème année id
Moyenne:19.0
Mention:Très bien
Admis: Oui
```

FIGURE 1 – Sortie console de l'Exercice 1

3 Exercice 2 : Vehicule / Voiture

Cet exercice met en évidence l'utilisation des constructeurs dans une relation d'héritage ainsi que l'importance de l'appel au constructeur de la classe mère à l'aide de `super()`.

3.1 Énoncé :

On souhaite modéliser un système de véhicules.

- La classe **Vehicule** contient :
  - un attribut : marque
  - des méthodes : afficherMarque(), demarrer(), arreter()
- La classe **Voiture** hérite de **Vehicule** et ajoute :
  - nombre de portes et carburant
  - des méthodes : afficherDetails(), klaxonner()

3.2 Vehicule.java :

```java
package Ex2;

public class Vehicule {
    String marque;

    public Vehicule(String marque) {
        this.marque = marque;
    }

    public void afficherMarque() {
        System.out.println("Marque est: " + marque);
    }

    public void demarrer() {
        System.out.println("La voiture est demarree");
    }

    public void arreter() {
        System.out.println("La voiture est arretee");
    }
}
```

3.3 Voiture.java :

```java
package Ex2;

public class Voiture extends Vehicule {

    int nombredeportes;
    String carburant;

    public Voiture(String marque, int nombredeportes, String carburant)
        {
        super(marque);
        this.nombredeportes = nombredeportes;
        this.carburant = carburant;
    }

    public void afficherDetails() {
        afficherMarque();
        System.out.println("Portes: " + nombredeportes);
        System.out.println("Carburant: " + carburant);
    }

    public void klaxonner() {
        System.out.println("pip pip pip pip");
    }
}
```

3.4 Main.java :

```java
package Ex2;

public class Main {
    public static void main(String[] args) {

        Voiture v1 = new Voiture("BMW", 4, "Essence");

        v1.demarrer();
        v1.afficherDetails();
        v1.klaxonner();
        v1.arreter();
    }
}
```

3.5 Résultat d'exécution :

```
la voiture est demarree
Marque est:BMW
Portes: 4
Carburant: Essence
pip pip pip pip
la voiture est arretee
```

FIGURE 2 – Sortie console de l'Exercice 2

4 Exercice 3 : Système bancaire

Cet exercice porte sur la conception d'un système de gestion de comptes bancaires en utilisant le principe d'héritage pour spécialiser les comportements selon le type de compte (Courant ou Épargne) et la redéfinition de méthodes.

4.1 Énoncé :

On souhaite concevoir un système de gestion de comptes bancaires.

- La classe **Compte** contient :
  - un attribut : solde
  - des méthodes : deposer(montant), retirer(montant), consulterSolde(), afficher()
- La classe **CompteCourant** hérite de **Compte** et ajoute :
  - un attribut : decouvertAutorise
  - des méthodes : autoriserDecouvert(), calculerFrais()
  - une redéfinition de la méthode afficher()

— La classe **CompteEpargne** hérite de **Compte** et ajoute :
  — un attribut : tauxInteret
  — des méthodes : calculerInterets(), ajouterInterets()
  — une redéfinition de la méthode afficher()

4.2 Compte.java :

```java
package Ex3;

public class Compte {
    double solde;

    public Compte(double solde) {
        this.solde = solde;
    }

    public void deposer(double montant) {
        solde += montant;
    }

    public void retirer(double montant) {
        if (montant <= solde) {
            solde -= montant;
        } else {
            System.out.println("Solde insuffisant !");
        }
    }

    public double consulterSolde() {
        return solde;
    }

    public void afficher() {
        System.out.println("Solde: " + solde);
    }
}
```

4.3 CompteCourant.java :

```java
package Ex3;

public class CompteCourant extends Compte {
    double decouvertAutorise;

    public CompteCourant(double solde, double decouvertAutorise) {
        super(solde);
        this.decouvertAutorise = decouvertAutorise;
    }

    public void autoriserDecouvert(double montant) {
        if (montant >= 0) {
            decouvertAutorise = montant;
```

```java
            System.out.println("Decouvert autorise fixe a " + decouvertAutorise + " DH");
        } else {
            System.out.println("Montant invalide.");
        }
    }

    public void calculerFrais() {
        double frais = solde * 0.02;
        solde -= frais;
    }

    public void afficher() {
        System.out.println("Compte Courant:");
        System.out.println("Solde: " + solde);
        System.out.println("Decouvert autorise: " + decouvertAutorise);
    }
}
```

4.4 CompteEpargne.java :

```java
package Ex3;

public class CompteEpargne extends Compte {
    private double tauxInteret;

    public CompteEpargne(double solde, double tauxInteret) {
        super(solde);
        this.tauxInteret = tauxInteret;
    }

    public double calculerInterets() {
        return solde * tauxInteret;
    }

    public void ajouterInterets() {
        solde += calculerInterets();
    }

    public void afficher() {
        System.out.println("Compte Epargne:");
        System.out.println("Solde: " + solde);
        System.out.println("Taux: " + tauxInteret);
    }
}
```

4.5 Main.java :

```java
package Ex3;

public class Main {
```

```java
    public static void main(String[] args) {

        CompteCourant cc = new CompteCourant(10000, 4500);
        cc.deposer(2000);
        cc.retirer(500);
        cc.calculerFrais();
        cc.afficher();

        System.out.println("--------------------");

        CompteEpargne ce = new CompteEpargne(20000, 0.02);
        ce.ajouterInterets();
        ce.afficher();
    }
}
```

4.6 Résultat d'exécution :

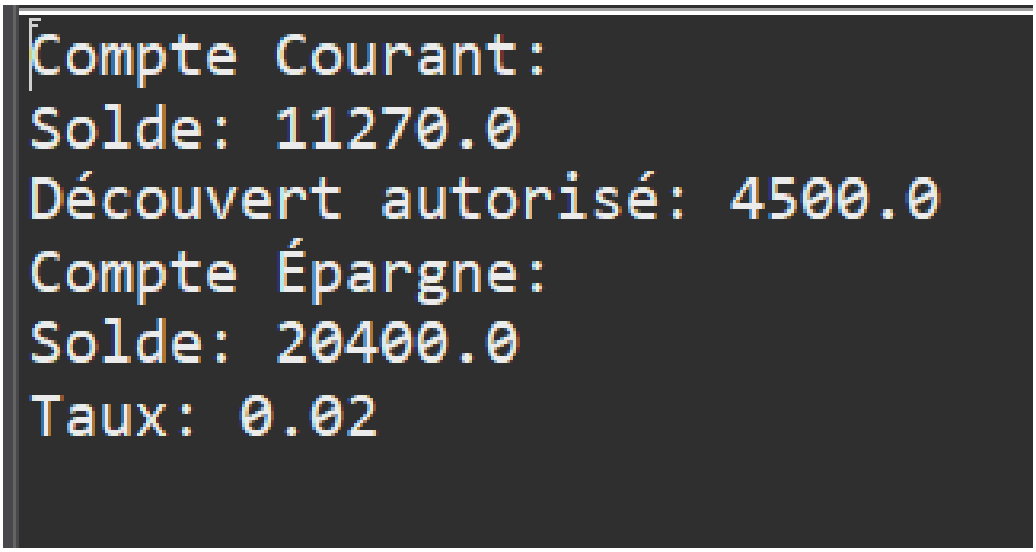

FIGURE 3 – Sortie console de l'Exercice 3 (Gestion Bancaire)

5 Exercice 4 : Polymorphisme - Animaux

Cet exercice met en pratique le concept de **polymorphisme** et la **redéfinition de méthodes** (Overriding). Il démontre comment une classe mère peut définir un comportement générique tandis que les classes filles spécialisent ce comportement.

5.1 Énoncé :

On souhaite modéliser des animaux avec des comportements dynamiques.

- La classe **Animal** contient les méthodes : crier(), manger(), et dormir().
- La classe **Chien** hérite d'Animal :
  - Redéfinit la méthode crier().
  - Ajoute les méthodes : garder() et jouer().

— La classe **Chat** hérite d'Animal :
  — Redéfinit la méthode crier().
  — Ajoute les méthodes : ronronner() et grimper().

5.2 Animal.java :

```java
package Ex4;

public class Animal {
    public void crier() {
        System.out.println("l'animal crie.");
    }
    public void manger() {
        System.out.println("l'animal mange.");
    }
    public void dormir() {
        System.out.println("l'animal dorme.");
    }
}
```

5.3 Chat.java :

```java
package Ex4;

public class Chat extends Animal {
    @Override
    public void crier() {
        System.out.println("Le chat miaule");
    }

    public void ronronner() {
        System.out.println("Le chat ronronne");
    }

    public void grimper() {
        System.out.println("Le chat grimpe");
    }
}
```

5.4 Chien.java :

```java
package Ex4;

public class Chien extends Animal {
    @Override
    public void crier() {
        System.out.println("Le chien aboie");
    }

    public void garder() {
```

```
        System.out.println("Le chien garde");
    }

    public void jouer() {
        System.out.println("le chien joue");
    }
}
```

5.5 Main.java :

```
package Ex4;

public class Main {
    public static void main(String[] args) {
        // Illustration du polymorphisme
        Animal a1 = new Chien();
        Animal a2 = new Chat();

        a1.crier(); // Appelle la m thode red finie du Chien
        a2.crier(); // Appelle la m thode red finie du Chat

        a1.manger();
        a2.dormir();

        // Utilisation des m thodes sp cifiques via instanciation
           directe
        Chien c = new Chien();
        c.crier();
        c.garder();

        Chat ch = new Chat();
        ch.crier();
        ch.ronronner();
    }
}
```

5.6 Résultat d'exécution :

```
Le chien aboie
Le chat miaule
l'animal mange.
l'animal dorme.
Le chien aboie
Le chien garde
Le chat miaule
Le chat ronronne
```

FIGURE 4 – Sortie console de l'Exercice 4 (Polymorphisme)

6 Exercice 5 : Polymorphisme - Figures Géométriques

Cet exercice illustre la manipulation uniforme de différentes formes géométriques au sein d'une application graphique. Il met en avant la capacité du polymorphisme à appeler des méthodes redéfinies tout en conservant des comportements communs (déplacement, redimensionnement) définis dans la classe mère.

6.1 Énoncé :

Dans une application graphique, différentes figures doivent être manipulées de manière uniforme.

- La classe **Figure** contient les méthodes : dessiner(), deplacer(x, y) et redimensionner(facteur).
- La classe **Cercle** hérite de Figure :
  - Attribut : rayon.
  - Redéfinit la méthode dessiner().
  - Ajoute les méthodes : calculerPerimetre() et calculerSurface().
- La classe **Triangle** hérite de Figure :
  - Attributs : côtés a, b, c.
  - Redéfinit la méthode dessiner().
  - Ajoute les méthodes : calculerPerimetre() et calculerSurface().

6.2 Figure.java :

```java
package Ex5;

public class Figure {
    public void dessiner() {
        System.out.println("Dessiner une figure");
    }

    public void deplacer(int x, int y) {
```

```
        System.out.println("Deplacer la figure vers (" + x + ", " + y +
            ")");
    }

    public void redimensionner(double facteur) {
        System.out.println("Redimensionner avec facteur: " + facteur);
    }
}
```

6.3 Cercle.java :

```
package Ex5;

class Cercle extends Figure {
    double rayon;

    public Cercle(double rayon) {
        this.rayon = rayon;
    }

    public void dessiner() {
        System.out.println("Dessiner un cercle");
    }

    public double calculerPerimetre() {
        return 2 * Math.PI * rayon;
    }

    public double calculerSurface() {
        return Math.PI * rayon * rayon;
    }
}
```

6.4 Triangle.java :

```
package Ex5;

class Triangle extends Figure {
    double a, b, c;

    public Triangle(double a, double b, double c) {
        this.a = a;
        this.b = b;
        this.c = c;
    }

    @Override
    public void dessiner() {
        System.out.println("Dessiner un triangle");
    }

```

```java
    public double calculerPerimetre() {
        return a + b + c;
    }

    public double calculerSurface() {
        double s = (a + b + c) / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }
}
```

6.5 Main.java :

```java
package Ex5;

public class Main {
    public static void main(String[] args) {
        // Utilisation du polymorphisme
        Figure f1 = new Cercle(2);
        Figure f2 = new Triangle(5, 6, 7);

        System.out.println("--- Test Cercle ---");
        f1.dessiner();
        f1.deplacer(2, 3);

        System.out.println("--- Test Triangle ---");
        f2.dessiner();
        f2.redimensionner(2);

        // Downcasting pour acceder aux methodes specifiques
        Cercle c = (Cercle) f1;
        System.out.println("Surface cercle: " + c.calculerSurface());

        Triangle t = (Triangle) f2;
        System.out.println("Perimetre triangle: " + t.calculerPerimetre
            ());
    }
}
```

6.6 Résultat d'exécution :

```
Cercle:
Dessiner un cercle
Déplacer la figure vers (2, 3)
Triangle:
Dessiner un triangle
Redimensionner avec facteur: 2.0
Surface cercle: 12.566370614359172
Périmètre triangle: 18.0
```

FIGURE 5 – Sortie console de l'Exercice 5 (Figures Géométriques)

7 Exercice 7 : Classe abstraite et calculs géométriques

Cet exercice met en œuvre l'utilisation d'une **classe abstraite** pour structurer des formes géométriques. La classe abstraite définit un contrat (méthode abstraite) que toutes les sous-classes doivent remplir, tout en fournissant des méthodes concrètes communes.

7.1 Énoncé :

On souhaite structurer les formes géométriques via une abstraction.

- La classe abstraite **Forme** contient :
  - Une méthode abstraite : `calculerSurface()`
  - Des méthodes concrètes : `afficherDescription()` et `comparerSurface(Forme f)`
- La classe **Rectangle** hérite de **Forme** et implémente :
  - Les méthodes : `calculerSurface()`, `calculerPerimetre()` et `validerDimensions()`.
- La classe **Cercle** hérite de **Forme** et implémente :
  - Les méthodes : `calculerSurface()`, `calculerPerimetre()` et `validerDimensions()`.

7.2 Forme.java :

```java
package Ex7;

public abstract class Forme {
    public abstract double calculerSurface();

    public void afficherDescription() {
        System.out.println("Ceci est une forme geometrique.");
    }

    public void comparerSurface(Forme f) {
        if (this.calculerSurface() > f.calculerSurface()) {
            System.out.println("Cette forme a une surface plus grande."
                );
        } else if (this.calculerSurface() < f.calculerSurface()) {
            System.out.println("L'autre forme a une surface plus grande
                .");
```

```
        } else {
            System.out.println("Les deux formes ont la meme surface.");
        }
    }
}
```

7.3 Cercle.java :

```
package Ex7;

public class Cercle extends Forme {
    double rayon;

    public Cercle(double rayon) {
        this.rayon = rayon;
    }

    public double calculerPerimetre() {
        return 2 * 3.14 * rayon;
    }

    public void validerDimensions() {
        if (rayon > 0) {
            System.out.println("Rayon valide");
        } else {
            System.out.println("Rayon invalide");
        }
    }

    @Override
    public double calculerSurface() {
        return 3.14 * rayon * rayon;
    }
}
```

7.4 Rectangle.java :

```
package Ex7;

public class Rectangle extends Forme {
    double longueur;
    double largeur;

    public Rectangle(double longueur, double largeur) {
        this.longueur = longueur;
        this.largeur = largeur;
    }

    public void validerDimensions() {
        if (largeur > 0 && longueur > 0) {
            System.out.println("Dimensions valides");
```

```java
        } else {
            System.out.println("Dimensions invalides");
        }
    }

    public double calculerPerimetre() {
        return 2 * (longueur + largeur);
    }

    public double calculerSurface() {
        return longueur * largeur;
    }
}
```

7.5 Main.java :

```java
package Ex7;

public class Main {
    public static void main(String[] args) {
        // Polymorphisme avec une classe abstraite
        Forme f1 = new Rectangle(5, 3);
        Forme f2 = new Cercle(2);

        System.out.println("--- Forme 1 (Rectangle) ---");
        f1.afficherDescription();
        System.out.println("Surface: " + f1.calculerSurface());

        System.out.println("--- Forme 2 (Cercle) ---");
        f2.afficherDescription();
        System.out.println("Surface: " + f2.calculerSurface());

        System.out.println("--- Comparaison ---");
        f1.comparerSurface(f2);

        // Downcasting pour acceder aux methodes specifiques de calcul
           de perimetre
        Rectangle r = (Rectangle) f1;
        System.out.println("Perimetre rectangle: " + r.
           calculerPerimetre());

        Cercle c = (Cercle) f2;
        System.out.println("Perimetre cercle: " + c.calculerPerimetre()
           );
    }
}
```

7.6 Résultat d'exécution :

```
Forme 1
Ceci est une forme géométrique.
Surface: 15.0
Forme 2
Ceci est une forme géométrique.
Surface: 12.56
Comparaison
Cette forme a une surface plus grande.
Périmètre rectangle: 16.0
Périmètre cercle: 12.56
```

FIGURE 6 – Sortie console de l'Exercice 7 (Abstraction)

8 Exercice 8 : Interface

Cet exercice explore l'utilisation des **interfaces** en Java. Contrairement à l'héritage de classe, une interface définit un contrat de comportement (`Volant`) que des classes de natures totalement différentes (`Avion` et `Oiseau`) peuvent implémenter à leur manière.

8.1 Énoncé :

On souhaite modéliser des entités capables de voler.

- L'interface **Volant** définit les signatures des méthodes : `voler()`, `atterrir()`, `changerAltitude()`.
- La classe **Oiseau** implémente **Volant** :
  - Elle fournit une implémentation spécifique pour les méthodes de vol.
  - Elle ajoute les méthodes : `migrer()` et `construireNid()`.
- La classe **Avion** implémente **Volant** :
  - Elle fournit une implémentation mécanique pour les méthodes de vol.
  - Elle ajoute les méthodes : `embarquerPassagers()` et `afficherAltitude()`.

8.2 Volant.java :

```java
package Ex8;

public interface Volant {
    void voler();
    void atterrir();
    void changerAltitude();
}
```

8.3 Avion.java :

```java
package Ex8;

public class Avion implements Volant {

```

```java
    public void voler() {
        System.out.println("L'avion decolle avec ses reacteurs");
    }

    public void atterrir() {
        System.out.println("L'avion descend sur la piste d'atterrissage
            ");
    }

    public void changerAltitude() {
        System.out.println("Le pilote ajuste l'altitude de l'avion");
    }

    public void embarquerPassagers() {
        System.out.println("Les passagers montent dans l'avion");
    }

    public void afficherAltitude() {
        System.out.println("Altitude actuelle: 10000 metres");
    }
}
```

8.4 Oiseau.java :

```java
package Ex8;

public class Oiseau implements Volant {

    public void voler() {
        System.out.println("L'oiseau vole");
    }

    public void atterrir() {
        System.out.println("L'oiseau se pose sur un arbre");
    }

    public void changerAltitude() {
        System.out.println("L'oiseau monte plus haut dans le ciel");
    }

    public void migrer() {
        System.out.println("L'oiseau part vers les pays chauds");
    }

    public void construireNid() {
        System.out.println("L'oiseau fabrique son nid");
    }
}
```

8.5 Main.java :

```java
package Ex8;

public class Main {
    public static void main(String[] args) {

        // Utilisation du polymorphisme via l'interface
        Volant v1 = new Oiseau();
        Volant v2 = new Avion();

        System.out.println("--- Test Oiseau ---");
        v1.voler();
        v1.changerAltitude();
        v1.atterrir();

        // Casting pour acceder aux methodes specifiques de l'Oiseau
        ((Oiseau) v1).migrer();
        ((Oiseau) v1).construireNid();

        System.out.println("\n--- Test Avion ---");
        v2.voler();
        v2.changerAltitude();
        v2.atterrir();

        // Casting pour acceder aux methodes specifiques de l'Avion
        ((Avion) v2).embarquerPassagers();
        ((Avion) v2).afficherAltitude();
    }
}
```

8.6 Résultat d'exécution :

```
Test Oiseau
L'oiseau vole
L'oiseau monte plus haut dans le ciel
L'oiseau se pose sur un arbre
L'oiseau part vers les pays chauds
L'oiseau fabrique son nid

 Test Avion
L'avion decolle avec ses reacteurs
Le pilote ajuste l'altitude de l'avion
L'avion descend sur la piste d'atterrissage
Les passagers montent dans l'avion
Altitude actuelle: 10000 metres
```

FIGURE 7 – Sortie console de l'Exercice 8 (Interfaces)

9 Exercice 9 : Héritage, Interface et Abstraction

Cet exercice combine les trois piliers de la programmation orientée objet : l'héritage, l'abstraction et les interfaces. Il illustre comment un objet peut à la fois hériter d'un comportement de base (`Vehicule`) et implémenter des capacités spécifiques (`Electrique`).

9.1 Énoncé :

On souhaite concevoir un système de véhicules modernes.

— La classe abstraite **Vehicule** définit l'attribut vitesse et les méthodes : `accelerer()`, `freiner()` et la méthode abstraite `demarrer()`.

— L'interface **Electrique** définit les signatures : `charger()` et `verifierBatterie()`.

— La classe **VoitureElectrique** hérite de Vehicule et implémente Electrique. [cite$_{s}tart]Elleajoutelesm$ `afficherAutonomie()`et`optimiserConsommation()`.

— La classe **Moto** hérite de Vehicule et ajoute les méthodes : `faireRoueArriere()` et `afficherTypeMoto()`.

9.2 Vehicule.java :

```java
package Ex9;

public abstract class Vehicule {
    protected double vitesse;

    public Vehicule(double vitesse) {
        this.vitesse = vitesse;
    }

    public void accelerer(double v) {
        vitesse += v;
        System.out.println("Vitesse apres acceleration: " + vitesse);
    }

    public void freiner(double v) {
        vitesse -= v;
        if (vitesse < 0) vitesse = 0;
        System.out.println("Vitesse apres freinage: " + vitesse);
    }

    public abstract void demarrer();
}
```

9.3 Electrique.java :

```java
package Ex9;

public interface Electrique {
    void charger();
    void verifierBatterie();
}
```

9.4 VoitureElectrique.java :

```java
package Ex9;

public class VoitureElectrique extends Vehicule implements Electrique {
```

```java
    private double batterie = 50; // %

    public VoitureElectrique(double vitesse) {
        super(vitesse);
    }

    public void demarrer() {
        System.out.println("La voiture electrique demarre silencieusement");
    }

    public void charger() {
        batterie = 100;
        System.out.println("Batterie chargee a 100%");
    }

    public void verifierBatterie() {
        System.out.println("Niveau batterie: " + batterie + "%");
    }

    public void afficherAutonomie() {
        System.out.println("Autonomie estimee: " + (batterie * 2) + " km");
    }

    public void optimiserConsommation() {
        System.out.println("Mode economie active");
    }
}
```

9.5 Moto.java :

```java
package Ex9;

public class Moto extends Vehicule {
    private String typeMoto = "Sport";

    public Moto(double vitesse) {
        super(vitesse);
    }

    public void demarrer() {
        System.out.println("La moto demarre avec bruit");
    }

    public void faireRoueArriere() {
        System.out.println("La moto fait une roue arriere !");
    }

    public void afficherTypeMoto() {
        System.out.println("Type de moto: " + typeMoto);
    }
}
```

9.6 Main.java :

```java
package Ex9;

public class Main {
    public static void main(String[] args) {
        // Utilisation du polymorphisme via la classe abstraite
        Vehicule v1 = new VoitureElectrique(0);
        Vehicule v2 = new Moto(0);

        System.out.println("--- Voiture Electrique ---");
        v1.demarrer();
        v1.accelerer(50);

        System.out.println("\n--- Moto ---");
        v2.demarrer();
        v2.accelerer(80);

        // Utilisation du polymorphisme via l'interface
        System.out.println("\n--- Services Electriques ---");
        Electrique e = (Electrique) v1;
        e.verifierBatterie();
        e.charger();

        // Acc s aux m thodes sp cifiques par casting
        VoitureElectrique ve = (VoitureElectrique) v1;
        ve.afficherAutonomie();

        Moto m = (Moto) v2;
        m.faireRoueArriere();
        m.afficherTypeMoto();
    }
}
```

9.7 Résultat d'exécution :

```
Voiture Électrique:
La voiture électrique démarre silencieusement
Vitesse après accélération: 50.0
Moto:
La moto démarre avec bruit
Vitesse après accélération: 80.0
Niveau batterie: 50.0%
Batterie chargée à 100%
Autonomie estimée: 200.0 km
La moto fait une roue arrière !
Type de moto: Sport
```

FIGURE 8 – Sortie console de l'Exercice 9 (Véhicules Modernes)

Voici la mise en forme pour l'exercice 10, structurée selon les sections du document PDF.

10 Exercice 10 : Système de Gestion de Bibliothèque (Synthèse)

Cet exercice final constitue une synthèse complète des concepts de programmation orientée objet. Il combine l'utilisation d'une classe mère abstraite, l'implémentation de plusieurs interfaces (`Empruntable`, `Consultable`) et la gestion des interactions entre différentes classes (`Document` et `Utilisateur`).

10.1 Énoncé :

On souhaite modéliser un système de gestion de bibliothèque intégrant différents types de supports et une gestion des utilisateurs.

- **Structure de base :** Une classe abstraite **Document** (identifiant, titre, auteur, disponibilité) avec une méthode abstraite `calculerDureeEmpruntMax()`.
- **Interfaces :**
  - **Empruntable** : pour les documents physiques (`estDisponible()`, `reserver()`).
  - **Consultable** : pour les documents numériques (`consulter()`, `afficherResume()`).
- **Spécialisations :**
  - **Livre** : hérite de Document et implémente Empruntable.
  - **Magazine** : hérite de Document et implémente Empruntable.
  - **DocumentNumerique** : hérite de Document et implémente Consultable.
- **Gestion des utilisateurs :** Une classe **Utilisateur** capable d'interagir avec n'importe quel type de `Document` via le polymorphisme.

10.2 Document.java :

```java
package Ex10;

public abstract class Document {
    String identifiant;
    String titre;
    String auteur;
    boolean disponible = true;

    public Document(String id, String t, String a) {
        this.identifiant = id;
        this.titre = t;
        this.auteur = a;
    }

    public void emprunter() {
        if (disponible) {
            disponible = false;
            System.out.println("Document emprunte: " + titre);
        } else {
            System.out.println("Impossible: " + titre + " n'est pas
                disponible");
        }
    }

    public void retourner() {
```

```java
        disponible = true;
        System.out.println("Document retourne: " + titre);
    }

    public void afficherInformations() {
        System.out.println("ID: " + identifiant + " | Titre: " + titre
            + " | Auteur: " + auteur);
    }

    public abstract int calculerDureeEmpruntMax();
}
```

10.3 Empruntable+Consultable.java :

```java
package Ex10;

public interface Empruntable {
    boolean estDisponible();
    void reserver();
}

interface Consultable {
    void consulter();
    void afficherResume();
}
```

10.4 Livre.java :

```java
package Ex10;

public class Livre extends Document implements Empruntable {
    int nombreDePages;
    String genre;

    public Livre(String id, String t, String a, int pages, String g) {
        super(id, t, a);
        this.nombreDePages = pages;
        this.genre = g;
    }

    public int calculerDureeEmpruntMax() {
        return 15;
    }

    public boolean estDisponible() {
        return disponible;
    }

    public void reserver() {
        System.out.println("Livre reserve.");
    }
```

```java

    public void afficherDetails() {
        afficherInformations();
        System.out.println("Pages: " + nombreDePages + " | Genre: " +
            genre);
    }
}
```

10.5 Magazine.java :

```java
package Ex10;

public class Magazine extends Document implements Empruntable {
    int numero;
    String datePublication;

    public Magazine(String id, String t, String a, int num, String date
        ) {
        super(id, t, a);
        this.numero = num;
        this.datePublication = date;
    }

    public int calculerDureeEmpruntMax() {
        return 3;
    }

    public boolean estDisponible() {
        return disponible;
    }

    public void reserver() {
        System.out.println("Magazine reserve.");
    }
}
```

10.6 DocumentNumerique.java :

```java
package Ex10;

public class DocumentNumerique extends Document implements Consultable
    {
    String format;
    double tailleFichier;

    public DocumentNumerique(String id, String t, String a, String f,
        double size) {
        super(id, t, a);
        this.format = f;
        this.tailleFichier = size;
    }
```

```java

    public int calculerDureeEmpruntMax() {
        return 0; // Les documents numeriques ne s'empruntent pas physiquement
    }

    public void consulter() {
        System.out.println("Ouverture du fichier " + format);
    }

    public void afficherResume() {
        System.out.println("Resume du document numerique de " + tailleFichier + " MB");
    }
}
```

10.7 Utilisateur.java :

```java
package Ex10;

public class Utilisateur {
    String identifiant;
    String nom;

    public Utilisateur(String id, String n) {
        this.identifiant = id;
        this.nom = n;
    }

    public void emprunterDocument(Document d) {
        System.out.println(nom + " essaie d'emprunter...");
        d.emprunter();
    }

    public void retournerDocument(Document d) {
        d.retourner();
    }
}
```

10.8 Main.java :

```java
package Ex10;

public class Main {
    public static void main(String[] args) {
        Livre l = new Livre("01", "Java POO", "MOHAMED", 300, "Technique");
        Magazine m = new Magazine("50", "Science", "Collectif", 120, "2026-04");
        DocumentNumerique d = new DocumentNumerique("9", "Cours POO", "Prof", "PDF", 2.5);
```

```java
        Utilisateur user = new Utilisateur("020", "HIDAYA");

        System.out.println("--- Test du Livre ---");
        l.afficherDetails();
        user.emprunterDocument(l);
        user.emprunterDocument(l); // Doit afficher que c'est
            impossible
        l.retourner();

        System.out.println("\n--- Test du Magazine ---");
        m.afficherInformations();
        System.out.println("Duree emprunt max: " + m.
            calculerDureeEmpruntMax() + " jours");
        user.emprunterDocument(m);

        System.out.println("\n--- Test du Document Numerique ---");
        d.consulter();
        d.afficherResume();
    }
}
```

10.9 Résultat d'exécution :

```
test du livre
ID: 01Titre:Java POOAuteur: MOHAMED
Pages: 300Genre: Technique
HIDAYA essaie d'emprunter
Document emprunte: Java POO
HIDAYA essaie d'emprunter
Impossible: Java POO n'est pas disponible
Document retourne: Java POO

 test du magazine
ID: 50Titre:Science Auteur: Collectif
Duree emprunt: 3 jours
HIDAYA essaie d'emprunter
Document emprunte: Science

 test du pdf
Ouverture du fichier PDF
Resume du document numerique de 2.5 MB
```

FIGURE 9 – Sortie console de l'Exercice 10 (Système de Bibliothèque)

11 Conclusion :

Pour conclure, ce TP nous a permis de mettre en pratique les concepts fondamentaux de la programmation orientée objet en Java. À travers ces dix exercices, nous avons appris à :

- Organiser le code avec l'**héritage** pour éviter les répétitions.
- Utiliser les **classes abstraites** et les **interfaces** pour créer des structures de code flexibles et modulaires (comme pour la gestion des véhicules ou de la bibliothèque).
- Maîtriser le **polymorphisme**, qui permet de manipuler différents types d'objets (animaux, formes, comptes bancaires) de manière uniforme.

L'application de ces notions nous a montré comment simplifier la conception de systèmes complexes et rendre le code plus facile à maintenir et à faire évoluer.